

**Proyecto final**

**CivilOps – Aplicación web para la gestión de obras de construcción**

Margarita María Arango Vélez

Facultad de Ingeniería

Universidad de Manizales

Programación I

Ing. Jose Ubaldo Carvajal

Mayo de 2026

# 1. Introducción

---

**CivilOps** es una aplicación web desarrollada en Python usando el framework Flask, diseñada para apoyar la gestión de obras de construcción. Permite registrar y consultar obras, empleados y materiales de manera organizada, simulando las necesidades reales de una empresa constructora.

Este proyecto fue desarrollado como trabajo final de la asignatura Programación I, con el objetivo de aplicar los conceptos de la Programación Orientada a Objetos (POO) estudiados durante el semestre, incluyendo clases, objetos, herencia, polimorfismo, encapsulamiento y composición.

La idea del sistema nace de mi experiencia profesional en el sector de la construcción, lo que me permitió diseñar un modelo básico de datos cercano a la realidad operativa de una empresa contratista.

## 2. Descripción del sistema

---

**CivilOps** está organizado en cuatro módulos principales, accesibles desde un menú lateral de navegación:

### 2.1 Dashboard

Pantalla principal que muestra un resumen ejecutivo del estado de la empresa: total de obras registradas, cuántas están en ejecución, presupuesto total de todas las obras y total de empleados. Desde aquí también se puede crear una nueva obra.

### 2.2 Obras

Lista todas las obras registradas con su código, nombre, ubicación, fechas, avance y estado. Permite ver el detalle de cada obra, editarla o eliminarla. En el detalle de la obra se pueden agregar, editar y eliminar empleados y materiales asignados a esa obra específica.

### 2.3 Empleados

Muestra el listado completo de todos los empleados de todas las obras, con su nombre, cédula, cargo, obra asignada, fecha de ingreso, salario y estado laboral (Activo o Retirado). Un mismo empleado puede estar asignado a varias obras diferentes.

### 2.4 Materiales

Lista todos los materiales registrados en todas las obras con su precio unitario, cantidad y costo total calculado automáticamente. Al final muestra la suma total de todos los materiales.

## 3. Conceptos de POO aplicados

---

A continuación se explica cómo se implementó cada concepto de Programación Orientada a Objetos en el proyecto, con ejemplos directos del código desarrollado.

### 3.1 Clases y Objetos

El proyecto define tres clases principales que representan entidades del mundo real en la construcción:

- Clase Obra: representa un proyecto de construcción
- Clase Trabajador: representa a una persona vinculada a la obra
- Clase Material: representa un insumo o recurso utilizado en la obra

En app.py se crean instancias (objetos) de estas clases con datos reales:

```
obra1 = Obra("Edificio Central", "Avenida Principal 123",
             codigo="OBR-2026-001", presupuesto=5000000000,
             fecha_inicio="01/02/2026", porcentaje_avance=45)
```

Cada vez que se crea una obra, un empleado o un material, se está instanciando un objeto de su clase correspondiente.

### 3.2 Método Constructor `__init__`

El método constructor `__init__` se define en cada clase para inicializar los atributos del objeto en el momento de su creación. Por ejemplo, en la clase Material:

```
def __init__(self, nombre, unidad, precio_unitario, cantidad):
    self.__nombre = nombre
    self.__unidad = unidad
    self.__precio_unitario = precio_unitario
    self.__cantidad = cantidad
```

La clase Obra tiene un constructor más completo que recibe nombre, ubicación, código, presupuesto, fechas, porcentaje de avance y estado, con valores por defecto para los campos opcionales.

### 3.3 Encapsulamiento

Todos los atributos de las clases están definidos como privados usando el prefijo doble guión bajo (`__`). Esto significa que no se pueden modificar directamente desde fuera de la clase — solo a través de métodos de acceso llamados getters y setters:

```
# Atributo privado
```

```

self.__salario = salario

# Getter: permite leer el valor
def get_salario(self):
    return self.__salario

# Setter: permite modificar el valor
def set_salario(self, salario):
    self.__salario = salario

```

El encapsulamiento protege la integridad de los datos. Por ejemplo, el setter del estado laboral valida que solo se acepten los valores 'Activo' o 'Retirado':

```

def set_estado_laboral(self, estado_laboral):
    if estado_laboral in ("Activo", "Retirado"):
        self.__estado_laboral = estado_laboral

```

### 3.4 Herencia

La clase Trabajador es la clase base (padre). A partir de ella se crean tres subclases (hijas) que heredan todos sus atributos y métodos:

- Obrero: representa personal de campo (oficiales, ayudantes)
- Ingeniero: representa personal técnico (ingenieros residentes, SISO)
- Arquitecto: representa personal de dirección (directores de obra, arquitectos residentes)

```

class Obrero(Trabajador):
    """Subclase Obrero que hereda de Trabajador."""
    def describir_rol(self):
        return f'Obrero {self.get_nombre()} trabaja con herramientas y mano de obra.'

```

Gracias a la herencia, las subclases no necesitan redefinir los atributos nombre, cédula, salario, fecha\_ingreso ni los métodos get/set — los heredan automáticamente de Trabajador.

### 3.5 Polimorfismo

El polimorfismo se implementa a través del método describir\_rol(). Este método está definido en la clase base Trabajador y es sobrescrito (override) en cada subclase con un comportamiento diferente según el tipo de trabajador:

| Clase      | Salida de describir_rol()                                                  |
|------------|----------------------------------------------------------------------------|
| Trabajador | Trabajador [nombre] con cédula [cédula].                                   |
| Obrero     | Obrero [nombre] (cédula [cédula]) trabaja con herramientas y mano de obra. |
| Ingeniero  | Ingeniero [nombre] (cédula [cédula]) supervisa la ejecución técnica.       |
| Arquitecto | Arquitecto [nombre] (cédula [cédula]) diseña y planifica la obra.          |

Aunque el método se llama igual en todas las clases, su comportamiento cambia dependiendo del tipo de objeto — eso es el polimorfismo en acción.

### 3.6 Composición

La composición se aplica en la clase Obra: una obra está compuesta por una lista de objetos Trabajador y una lista de objetos Material. Esto refleja la realidad — una obra tiene empleados asignados y materiales registrados:

```
def __init__(self, ...):
    self.__trabajadores = []    # lista de objetos Trabajador
    self.__materiales = []      # lista de objetos Material

def agregar_trabajador(self, trabajador):
    self.__trabajadores.append(trabajador)

def agregar_material(self, material):
    self.__materiales.append(material)
```

Además, un mismo objeto Trabajador puede estar referenciado en múltiples obras — por ejemplo, un Director de Obra que supervisa varios proyectos simultáneamente. Esto demuestra que la composición no implica exclusividad.

### 3.7 Métodos de instancia

Los métodos de instancia son funciones que operan sobre los datos del propio objeto. En este proyecto se implementaron varios métodos con lógica de negocio real:

| Clase    | Método                 | Qué hace                              |
|----------|------------------------|---------------------------------------|
| Material | calcular_costo()       | Multiplica precio_unitario × cantidad |
| Obra     | calcular_costo_total() | Suma costos de materiales y salarios  |

|            |                        |                                             |
|------------|------------------------|---------------------------------------------|
| Obra       | calcular_gasto_total() | Suma solo los costos de materiales          |
| Obra       | esta_en_ejecucion()    | Retorna True si el estado es 'En ejecución' |
| Obra       | esta_vencida()         | Compara la fecha de fin con la fecha actual |
| Trabajador | describir_rol()        | Describe el rol según el tipo de empleado   |

## 4. Tecnologías utilizadas

---

| Tecnología            | Uso en el proyecto                                                                                  |
|-----------------------|-----------------------------------------------------------------------------------------------------|
| Python 3              | Lenguaje principal. Se implementaron todas las clases y la lógica de negocio.                       |
| Flask                 | Framework web que permite levantar un servidor local y manejar las rutas de la aplicación.          |
| HTML / CSS            | Interfaz visual de la aplicación, con diseño responsivo y paleta de colores del sector constructor. |
| Tabler Icons          | Librería de íconos profesionales usados en la interfaz.                                             |
| GitHub                | Repositorio de control de versiones donde se almacena el código fuente del proyecto.                |
| VS Code + Claude Code | Entorno de desarrollo con asistencia de inteligencia artificial para la construcción del código.    |

## 5. Estructura del proyecto

---

El proyecto está organizado en los siguientes archivos y carpetas:

```
aplicativo_gestion_obras/
|
├── app.py                ← Servidor Flask y rutas de la aplicación
|
├── modelos/
|   ├── trabajador.py    ← Clase Trabajador + subclases (Herencia
y Polimorfismo)
|   ├── obra.py          ← Clase Obra (Composición)
|   └── material.py      ← Clase Material (Encapsulamiento)
|
└── templates/
    ├── base.html        ← Plantilla base con menú lateral
    ├── index.html       ← Dashboard principal
    ├── obras.html       ← Listado de obras
    ├── detalle_obra.html ← Hoja de vida de cada obra
    ├── trabajadores.html ← Listado de empleados
    └── materiales.html  ← Listado de materiales
```

## 6. Conclusiones

---

El desarrollo de **CivilOps** me permitió aplicar de manera práctica y en un contexto real todos los conceptos fundamentales de la Programación Orientada a Objetos estudiados durante el semestre.

La elección de un sistema de gestión de obras de construcción fue acertada porque las entidades del problema — obras, empleados y materiales — se mapean de forma natural a clases y objetos, lo que hizo que los conceptos de herencia, composición y polimorfismo aparecieran de manera natural en el diseño y no forzada.

El uso de Flask como framework web permitió ir más allá de una aplicación de consola y construir una interfaz visual funcional, lo que le da al proyecto un alcance cercano a un producto real que podría utilizarse en una empresa del sector.

El trabajo con Claude Code dentro de VS Code resultó ser una herramienta muy eficiente para el desarrollo, permitiendo construir y corregir código de manera ágil a través de prompts en lenguaje natural, lo que demostró que la inteligencia artificial puede ser un apoyo valioso en el proceso de aprendizaje de la programación.

## 7. Repositorio GitHub

---

El código fuente completo del proyecto está disponible en el siguiente repositorio público:

[https://github.com/hello-margarita/aplicativo\\_gestion\\_obras](https://github.com/hello-margarita/aplicativo_gestion_obras)